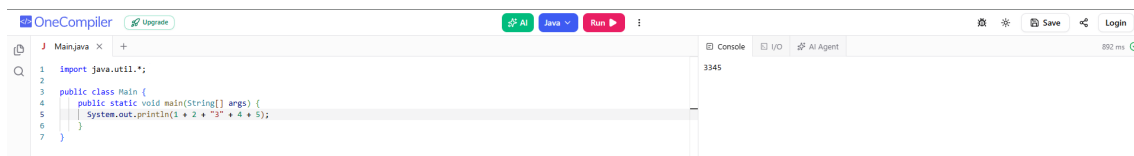


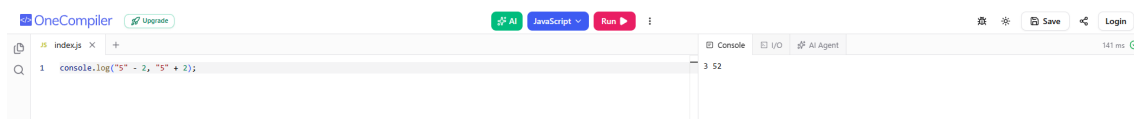
```
1 print((-2 ** 2, 2 ** 3 ** 2))
```

-4 512



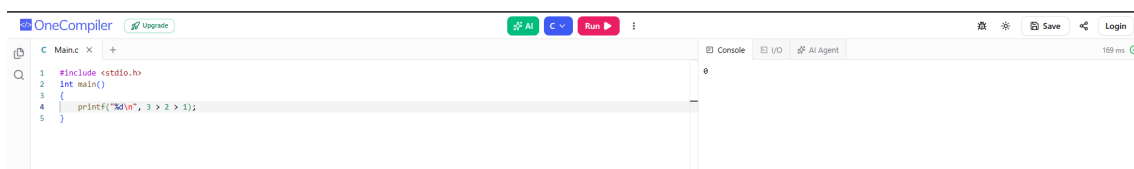
```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         System.out.println(3 * 2 + "3" * 4 * 5);
6     }
7 }
```

3345



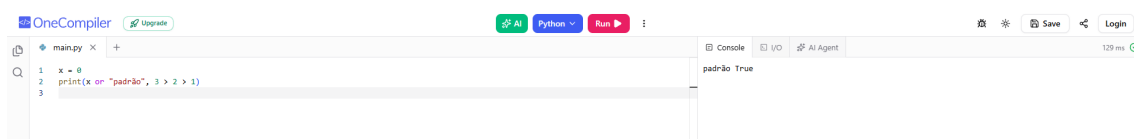
```
1 console.log("5" - 2, "5" * 2);
```

3 52



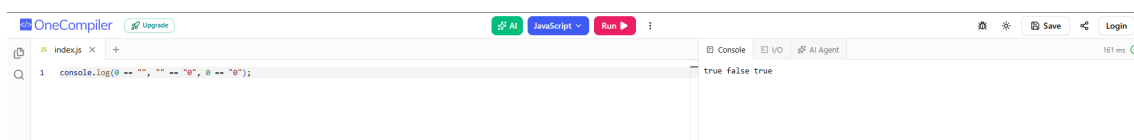
```
1 #include <stdio.h>
2 int main()
3 {
4     printf("%d\n", 3 > 2 > 1);
5 }
```

0



```
1 x = 0
2 print(x or "padrão", 3 > 2 > 1)
3
```

padrão True



```
1 console.log(0 == "", "" == "0", 0 == "0");
```

true false true

As linguagens possuem regras para avaliar expressões e controlar a execução do código. Entender essas regras evita erros de precedência, conversão de tipos e desvios. Casos reais, como o “goto fail” da Apple e o problema dos Zune, mostram como pequenos erros podem causar grandes problemas. Isso aparece no slide 2.

Nas expressões aritméticas, a precedência define qual operação acontece primeiro e a associatividade define a ordem entre operadores do mesmo nível. Como isso pode variar entre linguagens, usar parênteses ajuda a deixar a intenção clara. Slides 5 e 6.

A ordem de avaliação pode importar quando existem efeitos colaterais, como uma função alterando uma variável usada na mesma expressão. Java define uma ordem, enquanto C deixa algumas situações indefinidas. Sem efeitos colaterais, a ordem geralmente não faz diferença. Slide 7.

Sobrecarga de operadores permite que o mesmo símbolo tenha diferentes funções, como "+" para somar números ou juntar textos. É permitida em linguagens como Python, C++ e C#, mas não em Java. Slide 8.

Conversões podem ser de alargamento, estreitamento ou coerção. Go e Rust exigem mais conversões explícitas, enquanto JavaScript e PHP fazem muitas conversões automaticamente. Isso pode gerar resultados inesperados, como `"5" + 2` resultar em `"52"` no JavaScript. Slides 9 e 10.

Cada linguagem possui suas próprias regras para valores booleanos. C, por exemplo, permite usar números como valores lógicos, enquanto Java, C#, Go e Rust exigem `"boolean"`. O curto-circuito faz `"&&"` e `"||"` pararem de avaliar quando o resultado já é conhecido, evitando alguns erros. Slides 11 e 12.

Em C, a atribuição pode ser usada como expressão, permitindo `"a = b = 0"`, mas também causando erros como usar `"="` no lugar de `"=="`. Python, Go e Rust não tratam a atribuição dessa maneira. A divisão inteira também pode causar resultados inesperados. Slides 13 e 14.

No `"if"`, existe o problema do “else pendente”, quando não fica claro a qual `"if"` o `"else"` pertence. No `"switch"` do C, esquecer o `"break"` faz a execução continuar nos próximos casos. Slides 16 e 17.

Os laços repetem comandos enquanto uma condição for atendida ou enquanto houver elementos para percorrer. `"break"` interrompe o laço e `"continue"` pula para a próxima repetição. No JavaScript, `"for...in"` percorre chaves e `"for...of"` percorre valores. Slides 18 e 19.

O `"goto"` faz um desvio direto no código e pode dificultar sua compreensão, por isso comandos como `"break"`, `"continue"` e `"return"` são normalmente preferidos. Os comandos protegidos de Dijkstra permitem escolher entre condições verdadeiras, ideia relacionada ao `"select"` do Go. Slides 20 e 21.